development tool's Find and Replace feature makes this a relatively painless exercise, but that's not a sustainable way to build applications, especially when there are better options available for swapping implementations, which we will look at next.

7.1.3 Dependency inversion

So how do you achieve *loose coupling*? How can you redesign the consumer of your components or services, so they are no longer tightly coupled to a specific, or *concrete*, implementation? One solution is to rely on abstractions, rather than specific implementations. This approach is known as the *dependency inversion principle* (DIP), which is the *D* in the SOLID acronym. Dependency inversion is also known as *inversion of control* (IoC).

Abstract classes and interfaces represent abstractions in C#. As a rule of thumb, you will generally use interfaces as your abstraction, unless you have some common default behavior you want all implementations to share; in which case, you should choose an abstract class.

The dependency inversion principle states that "high-level modules should not depend on low-level modules. Both should depend on abstractions. Abstractions should not depend on details. Details should depend on abstractions" (Robert C. Martin: *Agile Software Development, Principles, Patterns, and Practices*, Pearson, 2002).

High-level modules tend to be consumers of services, and low-level modules tend to be the services themselves. So the first part of the DIP states that both the consumer and the service should depend on an abstraction, rather than the consumer depending on a specific service implementation. In the example, the abstraction will be an interface; the service will implement it, and the consumer will call it (figure 7.3).